

RESEARCHLY AI · R26-IT-116

Module 4

Research Performance Analytics

ARIMA trend forecasting, XGBoost quality & success scoring,
GCN-based concept mind maps, and cross-module dashboards —
architecture, ML methodology, and calculation logic

Module owner: H. W. S. S. Jayasundara

Service: `services/module4-analytics` (FastAPI, port 8004)

Frontend: `apps/web/src/app/(dashboard)/analytics/*`

Document generated: 27 August 2026

Contents

1. Overview	How the module works
2. System Architecture & Data Flow	Frontend → Gateway → ML service → DB
3. Feature A — Trend Forecasting	Per-topic ARIMA with confidence intervals
4. Feature B — Quality Scoring	XGBoost + calibrated heuristic blend
5. Feature C — Success Prediction	XGBoost + heuristic safety net
6. Feature D — Concept Mind Maps	KeyBERT seeds + GCN expansion
7. Feature E — Cross-Module Dashboard	Aggregated KPIs across Supabase tables
8. Planned vs. Production Models	Spec ensemble design vs. shipped code
9. Endpoint Reference	Full API surface

1 • Overview

Module 4 is the **analytics and foresight layer** of the platform — it turns the corpus and activity generated by the other three modules into forward-looking, decision-supporting numbers. It covers:

1. **Trend forecasting** — per-topic publication-count forecasts with confidence intervals, emerging-topic detection, and "where should I focus" recommendations.
2. **Quality scoring** — four-dimension quality score (originality, citation impact, methodology, clarity) for any proposal/paper text.
3. **Success prediction** — probability that a paper/proposal will be "successful" (publishable, methodologically sound), with actionable recommendations.
4. **Concept mind maps** — department/topic-level concept graphs for exploratory visualisation.
5. **Cross-module dashboard** — a single KPI snapshot aggregating data written by all four modules.

All five live in one FastAPI microservice (`services/module4-analytics` , port `8004`), consumed by pages under `apps/web/src/app/(dashboard)/analytics/` (`trends` , `quality-scores` , `predictions` , `mind-maps` , `dashboards`).

A recurring pattern in this module: every trained model (ARIMA, the two XGBoost regressor/classifier sets) is **blended with a hand-calibrated heuristic** rather than trusted alone at inference time. The code comments are explicit about why: the training heuristics the models learned from are narrow and can be unfair to legitimate technical/CS papers (e.g. penalising normal academic vocabulary length), so a calibrated heuristic acts as a fairness/safety net layered on top of the statistically-fit model rather than replacing it outright.

2 • System Architecture & Data Flow

```
Next.js page (analytics/*) | fetch() with Supabase JWT ▼ Express API Gateway (apps/api-gateway, port 3001) | requireAuth → mlRateLimiter → callMLService(4, path, method, body) ▼
FastAPI Module 4 service (services/module4-analytics, port 8004) | router → service layer
(trend_forecaster / quality_predictor / | success_predictor) → models/* ▼ ┌──
models/trained_trend_forecaster/trend_models.pkl (per-topic fitted ARIMA) ─┐
models/trained_quality_predictor/quality_models.pkl (per-dimension XGBoost regressors) ─┐
models/trained_success_predictor/success_model.pkl (XGBoost classifier) ─┐ models/mindmap/
(GCN weights + concepts.json, torch-geometric) ─┐ Supabase Postgres (research_proposals,
quality_scores, success_predictions, supervisor_matches, research_papers)
```

The gateway route file is `apps/api-gateway/src/routes/module4.routes.ts` , following the same `requireAuth → mlRateLimiter → callMLService(4, ...)` pattern as the other three modules (60-second proxy timeout).

3 • Feature A — Trend Forecasting

Per-topic ARIMA with 95% confidence intervals

What it does

GET `/trends` returns, per research topic, a historical year-by-year publication count and a forecast for the next N years (with lower/upper confidence bounds), a trend-direction label (rising/declining/stable), a projected growth percentage, a plain-English interpretation, and the model's in-sample accuracy (RMSE/MAE). `/trends/compare` ranks multiple topics by growth; `/trends/insights` surfaces "emerging" topics and auto-generated research-area recommendations.

Why ARIMA per topic

Publication-count-over-time is a classic univariate time series, and ARIMA (AutoRegressive Integrated Moving Average) is well suited to short, noisy annual count series like this — it natively models trend (via the differencing/integration term) and produces principled confidence intervals from the fitted model's own residual distribution, which a naive linear-regression forecast would not give for free. A separate ARIMA model is fit per topic (rather than one global model) because different research areas have genuinely different growth dynamics.

How it is implemented (`app/services/trend_forecaster.py`)

1. Loads `models/trained_trend_forecaster/trend_models.pkl` — a dict of fitted `statsmodels` ARIMA results objects, one per topic, each carrying its historical `years` / `counts` series.
2. **Forecast + CI** (`_forecast_with_ci`) — calls the fitted model's `get_forecast(steps=horizon)` , extracting the predicted mean and the 95% (`alpha=0.05`) prediction interval directly from `statsmodels` . Negative forecasts are clamped to 0 (a paper count can't be negative), and the upper bound is capped at $3 \times$ the historical maximum to guard against pathologically wide bands when residual variance is large. If the model object lacks `get_forecast()` , a heuristic fallback uses $\pm 1.96 \cdot \sigma$ from the residual standard deviation instead.
3. **Trend direction** (`_trend_direction`) — compares the mean of the last 3 years against the mean of everything before: $>1.2 \times$ earlier $\rightarrow$ "rising", $<0.8 \times$ $\rightarrow$ "declining", else "stable".
4. **Growth %** — $(\text{forecast_end} - \text{latest_historical}) / \text{latest_historical} \times 100$.
5. **Accuracy** (`_accuracy`) — RMSE and MAE computed against the model's own in-sample `fittedvalues` (aligned to account for ARIMA's differencing dropping the first d observations), normalised into an `nrmse` (RMSE / mean) for a scale-free 0..1 quality indicator.

Emerging-topic detection

```
recent_slope = least_squares_slope(counts[last 4 years]) long_slope =  
least_squares_slope(counts[all years]) emergence = recent_slope - long_slope growth_ratio =  
latest_count / mean(counts[:-3]) score = 0.5 * emergence + 0.5 * growth_ratio (kept only if  
recent_slope > 0 and latest_count ≥ 2 — filters out noise from thin/declining topics)
```

Recommendation scoring

```
growth_norm = clamp(growth_pct / 100, -1.0, 2.0) saturation = latest_count(topic) /  
max_count(across all topics) score = growth_norm - 0.4 · saturation (rewards high projected  
growth, penalises topics that are already heavily published in – "low competition")
```

4 • Feature B — Quality Scoring

XGBoost regressors + calibrated heuristic blend

What it does

`POST /quality/quality-score` takes a title + abstract (or a stored `proposal_id`) and returns four 0–1 dimension scores — *originality*, *citation_impact*, *methodology*, *clarity* — plus an overall weighted score, the paper's predicted topic, and targeted recommendations for improvement.

Why a model + heuristic blend, and why the blend ratio varies per dimension

The trained XGBoost regressors were fit on a training heuristic that turned out to be unevenly reliable across the four dimensions — the code's own comments explain that *methodology* and *originality* are the most biased in the model (it over-relies on narrow keyword/length signals), so the heuristic is given almost full authority there, while *citation_impact* and *clarity* get more weight from the trained model since those signals generalise better. This produces a fairer score for legitimate technical/CS papers than either the raw model or the raw heuristic alone.

How it is implemented (`app/services/quality_predictor.py`)

1. **Feature extraction** (`extract_features`) — word/sentence counts, average word/sentence length, a count of methodology keywords (25-term list, e.g. "experiment", "regression", "deep learning"), author count, and a citation-signal count (regex hits for `[1]` -style brackets, `(Author, Year)` parenthetical citations, and "et al." mentions in the abstract).
2. **Calibrated heuristic** (`_compute_quality_scores`) — deliberately designed to separate genuine research writing from generic documents (forms, rubrics, reports):

```
research_sig = min(1, count_of_matched_research_phrases / 4) e.g. "we propose", "this
paper", "our approach" strong_method = count of 45 strict research-specific methodology
terms (not generic words like "method") originality = min(1, 0.30·min(1,
abstract_len/1200) + 0.70·research_sig) citation_impact = min(1, citation_base + (0.25 if
multiple authors else 0)) citation_base: 0.70·min(1,signals/3) if signals>0, else 0.30 if
long research text, 0.15 if some research signal, else 0.05 methodology = min(1,
strong_method_count / 4) clarity = clamp(1 - max(0,(avg_word_len-7.5)/4) - max(0,
(avg_sent_len-30)/30), 0.30, 1.0) overall = 0.30·originality + 0.25·citation_impact +
0.25·methodology + 0.20·clarity
```

3. **Model blend** — if the trained bundle is available, each dimension's XGBoost prediction is blended with its heuristic counterpart at a *dimension-specific* ratio:

Dimension	Model weight	Heuristic weight	Rationale (from code comments)
methodology	0.10	0.90	Heuristic uses strict research-specific terms — near-fully authoritative
originality	0.15	0.85	Heuristic uses research-phrase detection, not just length — strongly authoritative

citation_impact	0.20	0.80	Model has some signal but heuristic still dominates
clarity	0.40	0.60	Model is comparatively more reliable here

`overall` is then recomputed from the blended dimension scores using the same 0.30/0.25/0.25/0.20 weights shown above, rather than blending the model's own overall output directly.

A companion function, `is_research_document()`, checks for the presence of standard academic sections (abstract/introduction/methodology/results/conclusion/references) before scoring, so a non-research document (e.g. an accidentally-uploaded form) can be flagged rather than silently scored as a weak paper.

5 • Feature C — Success Prediction

XGBoost classifier + heuristic safety net

What it does

POST `/predict/predict` (and `/predict/upload` for PDF/TXT files) returns a 0–1 `success_probability`, a binary `successful / needs_improvement` label, a confidence score, a *low/medium/high* risk level, and targeted improvement recommendations.

Why the same features feed a classifier here, not a regressor

This reuses the exact same `extract_features()` function as quality scoring (Section 4) but the trained artifact here is a binary XGBoost **classifier** — success is framed as a pass/fail outcome (publishable vs. not) rather than a continuous quality dimension, and `predict_proba` gives the probability of the positive class directly.

How it is implemented (`app/services/success_predictor.py`)

```
heuristic = _heuristic_score(features) – same structural formula as quality's overall score
(0.30/0.25/0.25/0.20 weights) but with thresholds recalibrated specifically to avoid
penalising technical/security papers: avg_word_length penalty starts at 7.5 (not 6.5),
methodology needs only 4 keyword hits (not 6), citation_impact gives 0.25 partial credit
even with zero abstract-citations model_proba = XGBoost.predict_proba(features)[1] (if
trained bundle available) success_probability = 0.40 · model_proba + 0.60 · heuristic
(heuristic dominates – code comment: "prevents extreme 0%/100% outputs for legitimate
technical papers")
```

The final label thresholds at 0.5 (successful / needs_improvement); risk level is ≥ 0.7 low, ≥ 0.4 medium, else high. Recommendations are generated directly from which raw feature fell short (e.g. *"Strengthen methodology section"* if fewer than 3 methodology keywords were found, *"Include citations..."* if fewer than 2 citation signals). For PDF uploads, `_extract_pdf_text` uses PyMuPDF to pull text, then a regex-based heuristic (`_META_RE`) skips article-ID/DOI/copyright header lines to find the real title, and searches for an explicit "Abstract" marker before falling back to the first ~3500 characters of body text (skipping the first 500, likely header/author block) if no abstract section is found or it's too short.

6 • Feature D — Concept Mind Maps

KeyBERT keyword extraction + 2-layer GCN expansion

What it does

`POST /mindmap/mindmap` builds a department- or topic-scoped concept graph — nodes are concepts with an importance weight and a domain cluster, edges carry a relationship-strength — for the frontend's D3 force-directed visualisation.

Why KeyBERT + a graph neural network

This is a two-stage pipeline: KeyBERT first extracts a small set of seed keyphrases from the free-text topic (using MMR-diversified extraction so the seeds themselves aren't redundant), then a trained 2-layer Graph Convolutional Network (GCN, via `torch_geometric`) expands those seeds into a richer concept neighbourhood by propagating information across a pre-built concept graph — the GCN captures relationships between concepts that keyword extraction alone cannot (e.g. that "convolutional neural network" and "image classification" are related even without shared vocabulary).

How it is implemented

1. **Seed extraction** (`app/routers/mindmap.py`) — if a topic is given, `KeyBERT().extract_keywords(topic, keyphrase_ngram_range=(1,2), top_n=5, use_mmr=True, diversity=0.5)` extracts up to 5 diverse 1–2-word seed concepts; on any import/runtime failure it falls back to simply splitting the topic string into its first 5 words.
2. **GCN expansion** (`app/models/mindmap_gnn.py::MindMapGNNModel`) — the `_GCNEncoder` is a standard 2-layer `GCNConv → ReLU → dropout(0.3) → GCNConv` stack producing concept embeddings; weights are loaded from `models/mindmap/model.pt` with dimensions read from a bundled `training_metadata.json`. Seed concepts are matched to the model's known `concepts.json` vocabulary via case-insensitive substring matching; nearby concepts (by index proximity within ± 3 — a proxy for graph adjacency baked in at training time) are added as related nodes, with edge weight decaying by 0.1 per index-step of distance ($0.8 - |\text{offset}| \cdot 0.1$). All seed nodes are additionally fully-connected to each other at a fixed weight of 0.9.
3. **Graceful degradation** — if `torch_geometric` isn't installed or no seed concept matches the model's vocabulary, the router falls back to a trivial star graph (all seed keywords connected directly to the first seed) rather than returning an empty response.

7 • Feature E — Cross-Module Dashboard

Aggregation only, no ML

What it does

`GET /analytics/dashboard` returns a single KPI snapshot: total proposals, average quality score, top-5 trending topics, count of "at-risk" projects, and count of active supervisors.

How it is implemented (`app/routers/dashboard.py`)

This endpoint is a pure Supabase aggregation layer that pulls from tables *written by every module* — it is the one place in the codebase that visibly demonstrates cross-module integration:

KPI	Source table	Computation
Total proposals	<code>research_proposals</code>	<code>count="exact"</code> row count
Avg quality score	<code>quality_scores</code> (written by Feature B)	mean of <code>overall_score</code>
Top trending topics	<code>research_papers.keywords</code> (written by Module 3's categorizer)	top-5 most frequent keywords, up to 500 papers sampled
At-risk projects	<code>success_predictions</code> (written by Feature C)	count where <code>success_probability < 0.5</code>
Active supervisors	<code>supervisor_matches</code> (written by Module 2)	distinct <code>supervisor_id</code> where <code>status = "active"</code>

Every query is individually wrapped in its own try/except, so a failure in one table (e.g. a table not yet populated) degrades that one KPI to zero rather than failing the whole dashboard.

8 • Planned vs. Production Models

Spec design	Status	What actually runs
ARIMA + Prophet ensemble , weighted by validation MAPE, target MAPE <22%, directional accuracy >75%	Only ARIMA is wired into the live forecaster	Per-topic ARIMA alone (Feature A) — accuracy is reported per-topic as RMSE/MAE/NRMSE rather than a single blended MAPE across a two-model ensemble
Success prediction — Random Forest (n=300, depth=8) + XGBoost (n=200, depth=6, lr=0.1) soft voting ensemble, target F1>0.75, ROC-AUC>0.80	Only XGBoost is present in the trained bundle	XGBoost classifier blended 40/60 with a calibrated heuristic (Feature C) — the RF half of the planned soft-voting ensemble is replaced by the heuristic safety net instead

Why this matters for the report: both simplifications follow the same underlying engineering judgement seen across the whole platform — a single well-tuned model blended with an explicit, inspectable heuristic is easier to calibrate, debug and justify to a viva panel than a second black-box model stacked on top of the first, while still meeting the spirit of the spec's ensembling intent (combining two independent signal sources rather than trusting one score blindly).

9 • Endpoint Reference

Method & Path (via gateway /api/v1/module4/...)	Feature	Backing service
GET /trends	Per-topic forecast	trend_forecaster.py
POST /trends/compare	Multi-topic comparison	trend_forecaster.py
GET /trends/insights	Emerging topics + recommendations	trend_forecaster.py
GET /trends/topics	Available topics list	trend_forecaster.py
POST /trends/report	HTML report download	trend_forecaster.generate_report_html
POST /quality/quality-score	Quality scoring	quality_predictor.py + topic_classifier.py
POST /predict/predict	Success prediction (text)	success_predictor.py
POST /predict/predict/upload	Success prediction (PDF/TXT)	PyMuPDF + success_predictor.py
POST /mindmap/mindmap	Concept mind map	KeyBERT + mindmap_gnn.py
GET /analytics/dashboard	Cross-module KPI snapshot	dashboard.py + Supabase